

Building Self-Improving Agents

How to give your agents persistent memory, cross-session learning, and a mechanism that makes them genuinely better over time.

CONTENTS

- 01 Why agents don't improve by default
- 02 The memory evolution: from files to file systems
- 03 In-band vs. out-of-band memory
- 04 Dreaming: the self-improvement loop

01

Why agents don't improve by default

Every time you start a new agent session, the agent starts from scratch. It has no memory of the last run. It will make the same mistakes. It will ask the same clarifying questions. It has no idea it has done this task fifty times before.

This is the core problem with raw model intelligence: it does not compound. Models get smarter with each release, but **the intelligence alone cannot accumulate task-specific knowledge** without a context layer that carries learning forward.

A newer model will not, out of the box, know your codebase conventions, your client's preferences, or the tool configuration that kept failing last week. That knowledge lives nowhere unless you build a system to capture it.

The solution is context engineering: systematically building, maintaining, and improving the information that agents load before they act. Done well, this multiplies model intelligence even as models continue to get smarter on their own.

Anthropic's applied AI team, working with frontier agent builders, identified a consistent progression. Each stage solves the failure mode of the previous one:

1 **CLAUDE.md / system prompt files**

Inject a markdown file at session start with codebase context, preferences, conventions.
Unreasonably effective for steering behavior. Breaks down when the file grows too large.

2 **Memory tools**

Let agents autonomously decide when to read, write, and update memories during a session. Works well. Still bounded by what the agent can see in one session.

3 **Skills (progressive disclosure)**

Store procedural workflows as files. Agent reads only the title before deciding to load a skill, keeping context lean. Like scanning a bookshelf before pulling a book.

4 **File system as memory**

Current best practice. Markdown files in a directory. Agents use bash and grep to search. No opinionated tools. Agents index and search intelligently.

02 + 03

The ceiling of in-band memory

Even a well-built file system memory hits a structural ceiling. The reason: all of the memory curation is happening **in-band**, meaning the agent is trying to do two things at once during a session.

IN-BAND MEMORY

Within the session

- Agent reads and writes memory while completing a task
- Fast feedback: improvements appear in the next session
- Competing objectives: task vs. memory curation fight for tokens
- Visibility limited to the current session only
- Cannot detect patterns that repeat across sessions
- Cannot see failures happening in other agents running in parallel

ALWAYS PRESENT

OUT-OF-BAND MEMORY (DREAMING)

Between sessions

- Separate async process with its own dedicated resources
- Analyzes transcripts from many sessions at once
- No competing objectives: fully focused on learning
- Full cross-session and cross-fleet visibility
- Spots systemic failures individual agents cannot see
- Human reviews proposed changes before they are applied

ADD WHEN SCALING

The frustration that surfaces in production: *"My agent keeps making the same mistake."* From the agent's perspective, it has never made this mistake before. Each session is a fresh context window. It has no way of knowing it is a recurring pattern.

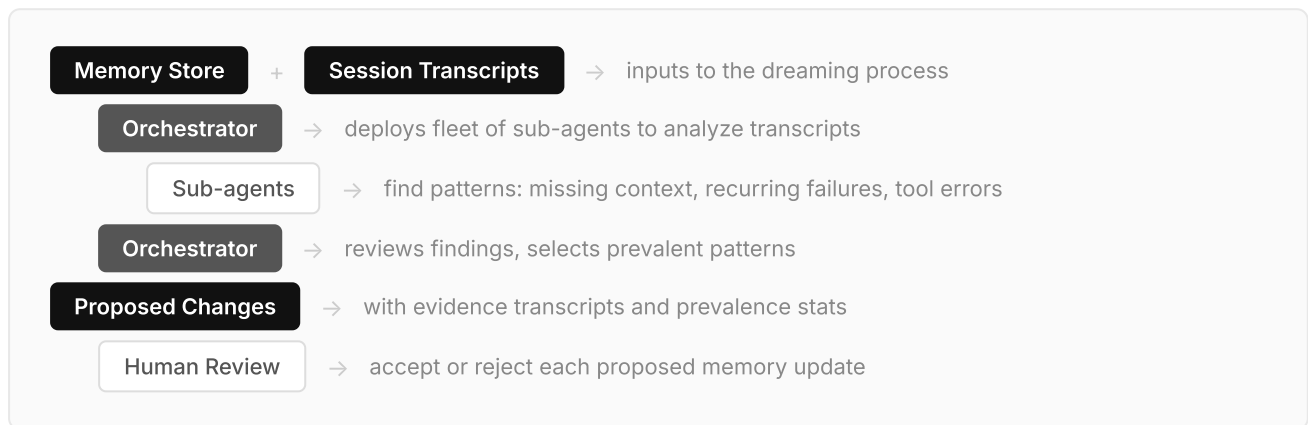
In-band memory gives you faster feedback. Out-of-band memory gives you cross-session visibility and dedicated learning capacity. You need both. They solve different problems at different timescales.

The optimal system runs them in parallel: in-band writes catch session-local lessons immediately, out-of-band dreaming catches fleet-wide patterns in batch and ensures memory quality over time.

04

How to implement Dreaming

Dreaming is a batch process that runs asynchronously. It takes your memory store and a set of session transcripts, analyzes them together, and proposes targeted improvements. Here is the exact mechanism:



Before you deploy dreaming at scale, add four production guardrails:

GUARDRAIL 01

Versioning

- Store every version of a memory update
- Track which session triggered the change
- Track which agent or human wrote it
- Enable rollback if an update is bad

GUARDRAIL 02

Concurrency Hashing

- Hash memory state before drafting an edit
- Hash again before committing
- If hashes differ: repull, redraft, retry
- Prevents silent corruption from simultaneous writes

GUARDRAIL 03

Permissioning Tiers

- Org-wide knowledge: read-only for agents
- Project context: limited write access
- Agent scratchpad: full write access
- One agent cannot overwrite the global context

GUARDRAIL 04

Portability

- Memory = hard-won organizational knowledge
- Design a clean API from day one
- Accessible across product surfaces and systems
- Avoid format lock-in before it is too late

Start simple: a CLAUDE.md file and basic memory writes. Add dreaming when agents run over long time horizons, in large fleets, or in complex domains where cross-session patterns matter.